

Interactive Maps with Leaflet

Visualisation and Sensing BSC2 2025 | Tom Armitage | t.armitage@arts.ac.uk

In this exercise, you'll use [Leaflet](#) to draw interactive ("slippy") maps in the browser with Javascript.

You'll start by putting points and features on a map manually; then, you'll look at working with JSON data, including GeoJSON formats.

Always prefer typing out code, rather than copy-pasting - unless the text tells you otherwise.

Timing

This workshop is designed to take an hour, including lecturer walkthrough.

Requirements

VS Code with Live Server extension

Getting Started

Open this directory in Visual Studio Code, and run [VS Code Live Server](#). You must access your code through a server running at localhost - you can't just double-click `index.html` in Explorer or Finder.

For each numbered section, I suggest starting with a new file. Start by basing your work on `index.html`

Throughout this workshop, we're going to refer to the [Leaflet Documentation](#) - this should be your first port of call when you run into something you don't know how to do.

1. Up and running with Leaflet

You should work in `index.html` , or in a copy of it in the same directory.

Start Live Server running in this directory.

At `http://localhost:5500` (or `http://localhost:5500/your_page.html` , if you've copied the index file to `your_page.html`), you should see a white page with the title "Maps Workshop".

Notes on the template HTML

The `index.html` template contains a few things already:

- it loads the Leaflet JS library
- it loads the CSS needed for Leaflet's styles to work
- it includes some simple CSS, primarily to set the element with id `map` to a fixed height. (Leaflet requires map divs to have set heights.)

Adding a map to the page.

Leaflet adds a map to a page by inserting it into a div that exist already.

Task: Create an empty `div` element in the page, after the `h1` ; give it the `id` "map".

All your remaining work will happen in Javascript, in the `script` block at the end of the file.

Task: Get Leaflet to put a map inside this element. In the script block, create a Leaflet map in the div:

```
const map = L.map("map");
```

The Leaflet library adds a global object `L` to your page, and all its functionality hangs off that. In this case, we're using the "map factory" to make a map - you can give it an `id` string or an `HTMLElement` object, as the [documentation tells you](#)

After you've done this, a big grey box will appear. This is your map - there's nothing to see because you've not added any tiles yet.

Task: add the OpenStreetMap tiles, which are readily available, to the map. Feel free to copy/paste this:

```
L.tileLayer("https://tile.openstreetmap.org/{z}/{x}/{y}.png", {
  maxZoom: 19,
  attribution:
    '&copy; <a href="http://www.openstreetmap.org/copyright">OpenStreetMap</a>'
}).addTo(map);
```

What's going on here?

- the tile layer takes a **template** as a string. This describes a pattern for tile URLs - the (z)oom level, and then x/y co-ordinate of the tile.
- the tiles have to be attributed manually. Tilesets will tell you how to attribute them.
- `tileLayer` return the newly created layer. It's Leaflet style to chain methods, so we call `addTo` on the newly created layer.

If you reload the browser at this point, nothing will appear. We need to set the starting view of the map.

Task: Call the `setView` method on your `map` object. The [documentation for the `setView` method on Map](#) tells us it takes a `LatLng`, a zoom level, and some optional options.

In Leaflet, a `LatLng` can be:

- an array of the format `[lat, lng]` . Eg: `[51, 0]`
- an object like so: `{lat: 51, lng: 0}`

Call `setView` on the map with a latitude and longitude of (0,0), and a zoomLevel of 2. You should see the world, centered on the equator.

Task: Now center it on (51.4739, -0.08013), zoom level 17. You should see something more familiar.

Adding a marker to the map.

The simplest element we can add to a map is a `Marker` , which describes a point.

Task: Add a marker to the map. Like the Tileset, you'll make a `Marker` object, then add it to the map. [Using the Marker documentation](#), add a `Marker` for the Greencoat building, at latitude 51.4723718881289, longitude -0.08614718914031984 .

Assign your marker to a variable when you create it. (`const marker = L.marker...`)

Task: Add a popup to the marker.

The marker appears, but it doesn't do anything. Let's add a popup.

The simplest way to add a popup is the `bindpopup` method, which you can call on a marker: `marker.bindpopup(text)`.

Bind a popup that says "Greencoat Building" to your marker.

Adding polygon features to the map.

Let's mark Peckham Road on the map. Here are four corners of the Peckham Road building, as latlng objects:

```
[
  { lat: 51.473791983443675, lng: -0.0812119245529175 },
  { lat: 51.47456716976322, lng: -0.0813567638397217 },
  { lat: 51.47490463900214, lng: -0.07923245429992677 },
  { lat: 51.47411943444423, lng: -0.07908225059509279 },
];
```

Task: [Using the Polygon documentation](#), create a polygon at these four corners.

You can also bind a popup to a polygon. Bind a popup to mark Peckham Road on your map.

2. Displaying data from GeoJSON.

Copy your work in progress to a new file (eg: `workshop2.html`). Work in this file.

Delete all your work after getting the map to appear on the screen and calling `setView` on it. In this exercise, you will import GeoJSON and display it on the map.

You'll fetch `/data/cci.geojson` using the [Fetch API](#), and use the built in GeoJSON parser to use it.

Fetch

The Fetch API is a Javascript interface inside modern browsers for making HTTP requests and parsing the response.

Making a GET request is simple - `fetch(url)`. `fetch` can also take an options object as a second parameter; this allows you to (eg) use other HTTP verbs, pass additional headers, pass form-encoded or JSON data, etc.

The fetch API returns its data as a [Promise](#). There are two ways to deal with promises: using `async/await`, and using `then/catch` methods. We'll use the latter, as it

might appear more familiar for now.

Task: Fetch GeoJSON data. You can fetch data in the browser like so:

```
fetch("/data/cci.geojson").then((data) => {  
  // data is your Response object  
  data.json().then((geoJson) => {  
    // geoJson is a Javascript object corresponding  
    // to the json in the file  
    //  
    // do something with that JSON data!  
  });  
});
```

Task: Using [Leaflet's GeoJSON method](#), add the data to your map:

```
L.geoJSON(geoJsonObjectName).addTo(map)
```

You should see the point and Polygon as before, as well as a line, describing a route between the two.

However, we've lost our popups. Let's add them back.

Have a look inside the `cci.geojson` file. You can see that each Feature has two properties: `name` and `desc`.

Task: Let's add a popup to each feature.

The `geoJSON` method can take an options object, as well as the JSON data to use.

One option, `onEachFeature`, can be set to a function. This function will be called for each feature in the dataset, and has two properties: the feature itself (ie, the data from the geojson file), and the Leaflet "layer" that is created for each feature.

Leaflet uses the idea of Layers to describe everything created in a Leaflet map - tilesets, markers, polygons, etc. Many Leaflet objects inherit behaviour from the Layer class - it's this class that affords the ability to `addTo` a map, amongst other things.

You can use the `onEachFeature` option to create a function that binds a popup to the layer that is created for each feature.

```
L.geoJSON(geoJsonObjectName, {
  onEachFeature: (feature, layer) => {
    layer.bindPopup(feature.properties.name);
  });
}).addTo(map);
```

Task: Also include the description in the popup. You can pass HTML into this method, which will help you format the name and description differently. Template strings may help you here.

3. Displaying LOTS of data

Let's now look at displaying lots of data.

The data we'll use isn't GeoJSON, but it *does* contain lat/lon information.

Create a copy of your GeoJSON-based file, say, `workshop3.html`.

Set the map view to focus on (0,0) at zoomlevel 3.

The data we'll use is a NASA dataset of meteorite landings on earth. Each item says where it landed, a rough (or accurate) date, whether it was found or someone saw it fall, and how heavy it was.

Instead of the CCI data, fetch `/data/meteorites.json`. You **won't** be able to pass it to `L.geoJson`, because it **isn't geoJSON**. Have a look at the file yourself!

You'll have to iterate over the data, and make a marker for each item. This dataset is very large, and if you use `L.marker`, you will probably crash your browser tab: by default, this creates DOM elements, and your browser won't like 400,000 DOM elements.

However, `circleMarker` just draws vector circles in SVG - much more performant than DOM elements - and the browser copes with that just fine! So we'll use that.

Task: With the loaded meteorites JSON:

- iterate over each item (the data is supplied as an array of objects).
- for each item, make a [CircleMarker](#) at `reclat, reclong`.
- when you've got them displaying, make a popup for each item that displays some of its items in an attractive manner.

This dataset is fun to see working - most `name` fields are related to the place it fell, so you'll be able to see if you've got it in the right place pretty quickly.

Clustering data

Let's find a better way of displaying many items at once, to avoid that 'red dot fever'.

Leaflet has a strong plugin ecosystem. We'll use the `markercluster` plugin to make it easier to read this map. I've included it in the workshop files for you.

Task: Add Markercluster to your page. At the end of the `<head>` element, add:

```
<link rel="stylesheet" href="/lib/leaflet-marker-cluster/MarkerCluster.css" />
<link
  rel="stylesheet"
  href="/lib/leaflet-marker-cluster/MarkerCluster.Default.css"
/>
<script src="/lib/leaflet-marker-cluster/leaflet.markercluster.js"></script>
```

Task: Now use this plugin to cluster your data.

The [documentation](#) shows how to use it in a simple manner.

You'll need to make a cluster group *before* you loop over all the meteorites; add each marker to the cluster group; and then finally, add the cluster group to the map *AFTER* the loop.

Try clicking on a marker in, say, Europe, and see how the plugin works.

You no longer need to use `circleMarker` - you can use regular `marker` now, because the clustering tool stops us needing so many DOM elements.

Although: there are 6186 markers clustered in the very middle of the map. (If you click this, things might grind to a halt). Why is this? Check the dataset to find out. What ways could you filter this data out? `